Programación por bloques

Nombre: _____ Fecha: _____

Qué vas a aprender

- Explicar qué es programar y qué es un algoritmo
- Representar algoritmos con diagramas de flujo
- Conocer el entorno de Scratch y crear programas con eventos y secuencias
- Usar bucles, condicionales, operadores, sensores y variables para resolver problemas
- Coordinar varios objetos con mensajes y localizar y corregir errores en los programas
- Relacionar la programación por bloques con el control de placas, robots y objetos físicos

Para pensar

Un ordenador puede ganar al campeón del mundo de ajedrez. ¿Significa eso que el ordenador es más listo que una persona?

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Rechazar Aceptar

Explicación

1.1 ¿Qué es programar?

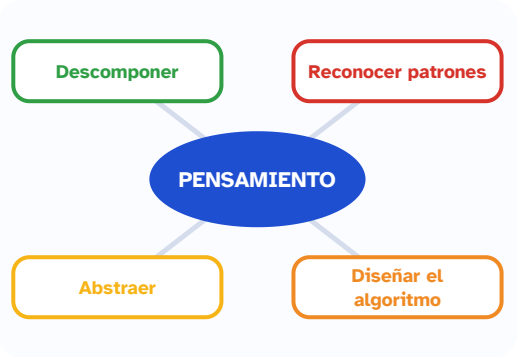
Programar es escribir instrucciones para que un ordenador u otra máquina realice una tarea. El conjunto de esas instrucciones, escritas en un orden concreto, forma un programa. Las aplicaciones del móvil, los videojuegos, las páginas web o el sistema que controla un ascensor son programas.

Un ordenador es muy rápido, pero no piensa por sí mismo: hace exactamente lo que le decimos, ni más ni menos. Si una instrucción está mal o falta, el resultado será incorrecto. Por eso, antes de programar hay que pensar bien cómo resolver el problema.

Esta forma de pensar se llama **pensamiento computacional**, y consiste en:

- **Descomponer** un problema grande en partes pequeñas.
- **Reconocer patrones** que se repiten.
- **Abstraer**: quedarse con lo importante y olvidar los detalles que no influyen.
- Diseñar un **algoritmo**: los pasos ordenados para resolverlo.

El pensamiento computacional no solo sirve para programar: ayuda a organizar cualquier tarea, desde preparar un examen hasta planificar un viaje.



1.2 Algoritmos y diagramas de flujo

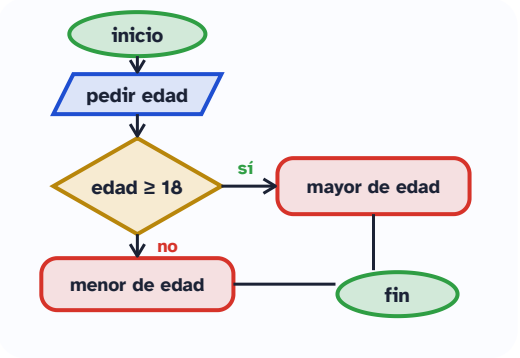
Un algoritmo es una secuencia ordenada y finita de pasos que resuelve un problema. Una receta de cocina, las instrucciones para montar un mueble o el

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Para representar un algoritmo de forma visual se usan los diagramas de flujo, con símbolos normalizados unidos por flechas:

- **Óvalo**: inicio y fin.
- **Rectángulo**: una acción o proceso («sumar los puntos»).
- **Romboide** o paralelogramo: entrada o salida de datos («pedir la edad», «mostrar el resultado»).
- **Rombo**: una decisión o pregunta con dos salidas, sí o no («¿edad ≥ 18?»).
- **Flechas**: indican el orden en que se siguen los pasos.

Dibujar el diagrama de flujo antes de programar ayuda a detectar errores y a pensar con claridad. Después, cada símbolo se traduce a instrucciones del lenguaje de programación.



1.3 Los lenguajes de programación

Los ordenadores, por dentro, solo entienden el **código máquina**: largas secuencias de ceros y unos, muy difíciles de manejar para las personas. Por eso se crearon los lenguajes de programación, que permiten escribir las instrucciones de una forma más comprensible y que luego se traducen al código máquina.

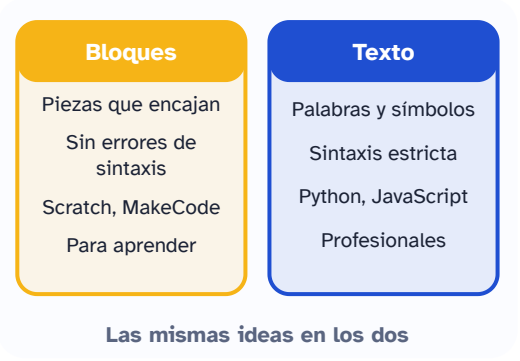
Hay dos grandes tipos:

- Los **lenguajes de texto**, en los que se escriben las instrucciones con palabras y símbolos siguiendo unas reglas muy estrictas (la **sintaxis**). Por ejemplo, Python, JavaScript, Java o C. Son los que usan los profesionales.
- Los **lenguajes de bloques** o visuales, en los que las instrucciones son piezas

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

para aprender. Los más conocidos son **Scratch**, creado en el MIT, MakeCode o App Inventor.

Las ideas fundamentales (secuencias, bucles, condicionales, variables) son las mismas en todos los lenguajes. Lo que se aprende con bloques sirve después para programar con texto.



1.4 El entorno de Scratch

Scratch es un lenguaje de programación por bloques gratuito que se usa en el navegador o instalado en el ordenador. Su pantalla tiene varias zonas:

• El **escenario**: el rectángulo donde ocurre la acción. Mide 480 × 360 pasos y usa coordenadas: el centro es el punto (0, 0); la x va de –240 a 240 y la y, de –180 a 180.
• Los **objetos** o *sprites*: los personajes y elementos que se mueven en el escenario. Cada objeto tiene sus propios programas, **disfraces** (imágenes) y **sonidos**. El escenario también puede tener **fondos** y programas.
• La **paleta de bloques**, organizada por categorías y colores: Movimiento (azul), Apariencia (morado), Sonido (rosa), Eventos (amarillo), Control (naranja), Sensores (azul claro), Operadores (verde) y Variables (naranja oscuro).
• El **área de programas**: donde se arrastran y encajan los bloques.

Los proyectos se pueden guardar en la cuenta de Scratch y **compartir** con la comunidad, donde otras personas pueden verlos, comentarlos y hacer «remezclas» citando el original.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Categoría	Color	Ejemplo
Movimiento	Azul	mover 10 pasos
Apariencia	Morado	decir Hola
Eventos	Amarillo	al hacer clic
Control	Naranja	repetir 10
Sensores	Celeste	¿tocando borde?

1.5 Eventos y secuencias

Todo programa en Scratch empieza con un *evento*: algo que ocurre y pone en marcha las instrucciones. Los bloques de eventos tienen forma de sombrero, porque van siempre arriba. Los más usados son «al hacer clic en la bandera verde», «al presionar la tecla espacio», «al hacer clic en este objeto» o «al recibir mensaje».

Debajo del evento se encajan las instrucciones, que se ejecutan una detrás de otra, de arriba abajo: es una *secuencia*. El orden es fundamental: no es lo mismo «mover 100 pasos y luego girar» que «girar y luego mover».

Un ejemplo de secuencia: al hacer clic en la bandera, el gato va a la posición (0, 0), dice «¡Hola!» durante 2 segundos, se mueve 100 pasos y maúlla. Es buena costumbre empezar los programas **inicializando**: colocando cada objeto en su posición, con su disfraz y su tamaño iniciales, para que el programa funcione igual cada vez que se ejecute.

Un mismo objeto puede tener varios programas, cada uno con su evento, que funcionan a la vez.

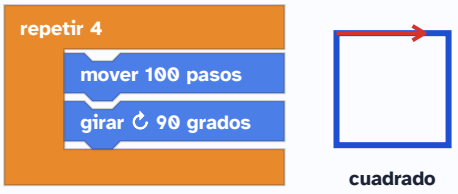


Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Muchas veces hay que repetir las mismas instrucciones. En lugar de copiarlas una y otra vez, se usa un bucle: un bloque que repite lo que tiene dentro. Los bloques de bucle tienen forma de C y están en la categoría Control. Scratch tiene tres:

• **Repetir n veces**: repite las instrucciones un número fijo de veces. Por ejemplo, para dibujar un cuadrado con el lápiz: repetir 4 veces «mover 100 pasos» y «girar 90 grados». Para un triángulo, repetir 3 veces girando 120 grados; para un hexágono, 6 veces girando 60 grados. El ángulo es siempre 360 dividido entre el número de lados.
• **Por siempre**: repite sin parar mientras el programa esté en marcha. Se usa en los juegos para comprobar continuamente lo que pasa, por ejemplo, si el personaje toca un enemigo.
• **Repetir hasta que**: repite hasta que se cumpla una condición, por ejemplo, «hasta que toque el borde».

Los bucles hacen los programas más cortos, más claros y más fáciles de modificar: para hacer un cuadrado más grande, basta con cambiar un número.



4 lados · 4 giros de 90°

1.7 Los condicionales

Los programas necesitan tomar **decisiones** según lo que ocurra. Para eso están los condicionales, que ejecutan unas instrucciones solo si se cumple una **condición**. Una condición es una pregunta cuya respuesta solo puede ser verdadero o falso, y en Scratch se representa con bloques de forma hexagonal.

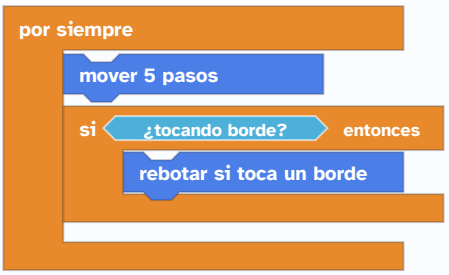
Hay dos bloques condicionales:

• **Si … entonces**: si la condición es verdadera, ejecuta lo que tiene dentro; si es

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

instrucciones; si es falsa, ejecuta el segundo. Siempre se hace una de las dos cosas.

Los condicionales casi siempre van dentro de un bucle «por siempre», para comprobar la condición continuamente. Por ejemplo: por siempre, si el gato toca el borde, entonces rebota y dice «¡Ay!». O bien: si la respuesta es igual a 12, entonces decir «¡Correcto!», si no, decir «Inténtalo otra vez». Corresponden al rombo de los diagramas de flujo.



1.8 Sensores y operadores

Los bloques de sensores permiten que el programa «perciba» lo que pasa: si un objeto toca a otro, toca un color o el borde; si se presiona una tecla o el ratón; dónde está el puntero; cuánto tiempo ha pasado (cronómetro); o el volumen del micrófono. También permiten **preguntar** algo al usuario y guardar su **respuesta**.

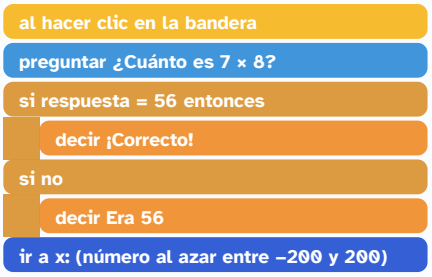
Los operadores (en verde) permiten hacer cálculos y construir condiciones más complejas:

- **Aritméticos**: sumar, restar, multiplicar y dividir.
- **De comparación**: mayor que (>), menor que (<) e igual a (=).
- **Lógicos**: «y» (se cumplen las dos condiciones), «o» (se cumple al menos una) y «no» (lo contrario).
- **Números al azar**: «número aleatorio entre 1 y 10», muy útil en los juegos para que no sean siempre iguales.
- De **texto**: unir palabras, contar letras.

Por ejemplo, para que un juego termine cuando el personaje toca un enemigo **y**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Preguntar y comprobar



1.9 Las variables

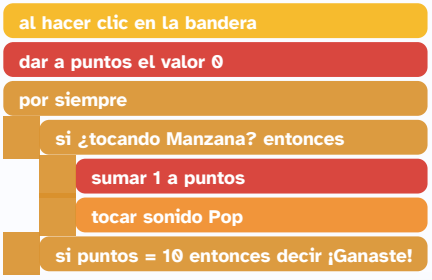
Una variable es un espacio de la memoria con un **nombre** que guarda un **valor** que puede cambiar mientras funciona el programa. Es como una caja con una etiqueta: la etiqueta es el nombre («puntos») y lo de dentro, el valor (0, 1, 2…).

En Scratch, las variables se crean en la categoría Variables y se manejan con tres bloques:

- **Dar a** la variable un valor: se usa al principio para inicializarla, por ejemplo, «dar a puntos el valor 0».
- **Sumar a** la variable una cantidad: «sumar 1 a puntos» cada vez que el jugador atrapa una manzana. Para restar, se suma un número negativo.
- **Mostrar** u ocultar la variable en el escenario.

Las variables sirven para contar puntos, vidas, tiempo, nivel, velocidad o para guardar la respuesta de un usuario. Hay que darles un **nombre descriptivo** y no olvidar **inicializarlas**: si no se pone a 0 al empezar, la puntuación de la partida anterior seguirá ahí. Además, las **listas** permiten guardar muchos valores bajo un mismo nombre, como los nombres de los jugadores.

Contar puntos

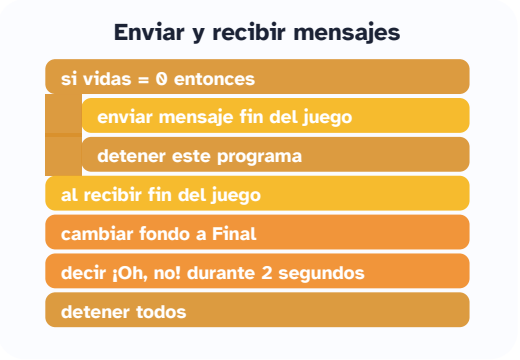


Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

En un juego o una historia suele haber varios objetos que deben actuar de forma **coordinada**: un personaje habla y otro le responde, o al acabar la partida todos los enemigos desaparecen. Para comunicarlos se usan los mensajes.

Un objeto puede **enviar** un mensaje con el bloque «enviar mensaje», y todos los objetos que tengan un programa que empiece por «al recibir» ese mensaje se ponen en marcha. Es como un aviso por megafonía: quien lo emite no sabe quién lo escucha, pero quien debe reaccionar, reacciona. Por ejemplo, cuando la variable vidas llega a 0, el escenario envía el mensaje «fin del juego»; al recibirlo, el personaje dice «¡Oh, no!», los enemigos se esconden y el fondo cambia a una pantalla de final.

También se pueden crear **clones**: copias de un objeto que se generan mientras funciona el programa, muy útiles para hacer muchas gotas de lluvia, disparos o enemigos sin tener que dibujarlos uno a uno. Cada clon funciona con sus propios programas «al comenzar como clon».



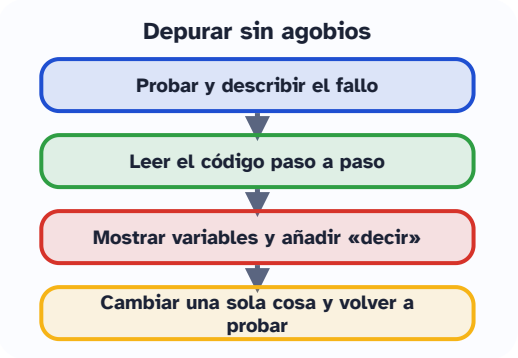
1.11 Depurar: encontrar y corregir errores

Es muy raro que un programa funcione bien a la primera. A los errores de programación se les llama *bugs* («bichos», en inglés), y el proceso de buscarlos y corregirlos se llama depurar. Es una parte normal del trabajo de cualquier programador.

Errores frecuentes en Scratch: olvidar el evento de inicio, no inicializar posiciones o variables, poner los bloques en el orden equivocado, colocar una instrucción fuera del bucle en lugar de dentro, usar «repetir» en lugar de «por siempre», confundir el nombre de un mensaje o hacer movimientos tan rápidos que no se

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

• **Ejecutar el programa por partes**: hacer clic en un bloque para ver qué hace solo.
• **Leer el código en voz alta** paso a paso, como si fueras el ordenador.
• **Mostrar las variables** en el escenario para ver cómo cambian.
• **Añadir bloques «decir»** que indiquen por dónde va el programa.
• **Cambiar una cosa cada vez** y volver a probar.
• Explicárselo a un compañero: a menudo, al contarlo, se descubre el fallo.



1.12 Programar objetos del mundo real

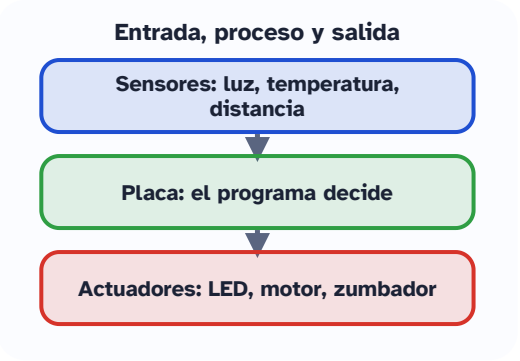
La programación por bloques no solo sirve para juegos en la pantalla: también permite controlar objetos físicos. Para ello se usan placas programables, pequeños ordenadores con entradas y salidas, como la **micro:bit**, Arduino o las placas de los kits de robótica.

Estos sistemas tienen tres partes:

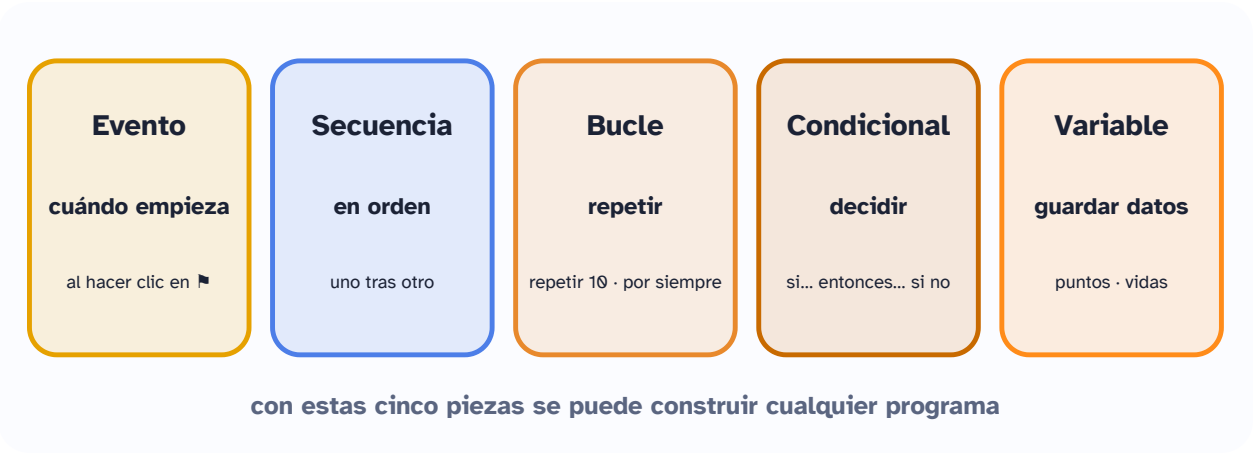
• Los **sensores** o entradas, que recogen información del entorno: botones, sensores de luz, de temperatura, de distancia, de movimiento (acelerómetro) o de sonido.
• La **placa**, que ejecuta el programa y decide qué hacer.
• Los **actuadores** o salidas, que actúan sobre el entorno: LED, pantallas, zumbadores, motores o servomotores.

Con un programa de bloques (por ejemplo, MakeCode para la micro:bit) se pueden crear un termómetro que muestre la temperatura, un podómetro, una alarma que suene si alguien abre una caja, un semáforo con LED o un pequeño

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información



Las estructuras básicas de la programación



Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Ejercicios

1. **Define programa y algoritmo.**

2. **Escribe un algoritmo de la vida diaria con al menos cinco pasos.**

3. **Dibuja y nombra los símbolos del diagrama de flujo para inicio, acción, entrada de datos y decisión.**

4. **Escribe dos lenguajes de bloques y dos de texto.**

5. **¿Qué son el escenario y los objetos en Scratch?**

6. **¿Para qué sirve un bloque de evento? Escribe dos ejemplos.**

7. **¿Qué es un bucle? Nombra los tres bucles de Scratch.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

8. **¿Qué es un condicional?**

9. **¿Qué es una variable? Pon dos ejemplos de uso en un juego.**

10. **¿Qué significa depurar un programa?**

11. **Programa en Scratch un cuadrado de lado 80 y un hexágono de lado 50. Indica los giros.**

12. **Dibuja el diagrama de flujo de un programa que pregunta la nota de un examen y dice «aprobado» si es 5 o más y «suspenso» si no.**

13. **Traduce el diagrama anterior a bloques de Scratch.**

14. **Explica la diferencia entre «repetir 10 veces» y «por siempre», con un ejemplo de cuándo usar cada uno.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

15. **¿Qué hacen los operadores lógicos «y», «o» y «no»? Pon un ejemplo de cada uno.**

16. **¿Por qué hay que inicializar las variables al principio del programa?**

17. **Programa que la variable tiempo empiece en 30 y baje 1 cada segundo hasta llegar a 0, y entonces se diga «¡Se acabó!».**

18. **Explica para qué sirven los mensajes con un ejemplo.**

19. **¿Qué son los clones en Scratch y cuándo son útiles?**

20. **Escribe cuatro estrategias para depurar un programa.**

21. **Explica qué son los sensores y los actuadores en una placa como la micro:bit.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

22. **Describe el pensamiento computacional aplicado a organizar una fiesta de cumpleaños.**

23. **Diseña un juego de atrapar: una manzana cae desde arriba en una posición aleatoria y el jugador mueve una cesta con las flechas. Cada manzana atrapada suma un punto; si cae al suelo, se pierde una vida. Describe los programas.**

24. **Escribe un programa que pregunte cinco tablas de multiplicar al azar y cuente los aciertos.**

25. **Dibuja el diagrama de flujo de un semáforo que repite: verde 5 s, ámbar 2 s, rojo 5 s.**

26. **Un programa debería contar los puntos, pero siempre muestra 1. Programa: por siempre, si tocando estrella entonces dar a puntos el valor 1. Encuentra el error y corrígelo. ¿Hay otro problema?**

27. **Programa con bloques una micro:bit como termómetro que muestre una cara feliz si la temperatura está entre 18 y 25 °C y una triste en**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

28. **Explica por qué los programadores dividen un programa grande en partes (bloques personalizados o funciones).**

29. **Compara un algoritmo de un videojuego con el de un robot aspirador: ¿qué entradas, procesos y salidas tiene cada uno?**

30. **Reflexiona: ¿qué cuestiones éticas plantean los programas que deciden por las personas (por ejemplo, qué vídeos vemos o quién consigue un trabajo)?**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Practica

1. **Un programa es...**

   A) un componente del ordenador
   B) un conjunto de instrucciones para un ordenador
   C) una red de ordenadores
   D) un tipo de pantalla

2. **Un algoritmo es...**

   A) una secuencia ordenada y finita de pasos
   B) un virus informático
   C) un lenguaje de bloques
   D) una placa programable

3. **En un diagrama de flujo, la decisión se dibuja con un...**

   A) óvalo
   B) rectángulo
   C) círculo
   D) rombo

4. **El inicio y el fin se representan con un...**

   A) rombo
   B) rectángulo
   C) óvalo
   D) romboide

5. **Scratch es un lenguaje...**

   A) de texto
   B) de código máquina
   C) de bloques
   D) de hojas de cálculo

6. **En Scratch, el centro del escenario es el punto...**

   A) (240, 180)
   B) (0, 0)
   C) (−240, −180)

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

A) objetos o sprites
B) fondos
C) variables
D) mensajes

8. **Los bloques de eventos son de color...**

A) azul
B) verde
C) rosa
D) amarillo

9. **Una secuencia es...**

A) una repetición
B) una decisión
C) una variable
D) instrucciones que se ejecutan una tras otra

10. **Para dibujar un cuadrado se usa repetir 4 veces mover y girar...**

A) 45 grados
B) 60 grados
C) 90 grados
D) 120 grados

11. **Para un hexágono se gira...**

A) 90 grados
B) 60 grados
C) 30 grados
D) 120 grados

12. **El bucle que no termina nunca es...**

A) por siempre
B) repetir 10
C) repetir hasta que
D) si ... entonces

13. **El bloque «si ... entonces ... si no»...**

A) siempre ejecuta una de las dos opciones

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

D) crea variables

14. **Una condición solo puede valer...**

A) cualquier número
B) un texto
C) un color
D) verdadero o falso

15. **El operador «y» es verdadero si...**

A) se cumple una de ellas
B) no se cumple ninguna
C) se cumplen las dos condiciones
D) siempre

16. **Una variable sirve para...**

A) dibujar con el lápiz
B) guardar un valor que puede cambiar
C) cambiar de fondo
D) tocar sonidos

17. **Al empezar un juego, la variable puntos debe...**

A) borrarse
B) inicializarse a 0
C) ocultarse siempre
D) valer 100

18. **Para que un objeto avise a otros se usan los...**

A) mensajes
B) disfraces
C) sonidos
D) lápices

19. **Buscar y corregir errores se llama...**

A) compilar
B) descargar
C) clonar
D) depurar

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

B) algoritmo

C) actuador

D) evento

Explícalo con tus palabras

Incluye: Qué es un algoritmo y cómo se dibuja un diagrama de flujo; Para qué sirven los eventos y las secuencias en Scratch; Qué diferencia hay entre un bucle y un condicional; Para qué sirve una variable y por qué hay que inicializarla; Cómo se depura un programa que no funciona.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Resumen



Glosario

Programa	Conjunto de instrucciones ordenadas que ejecuta un ordenador.
Algoritmo	Secuencia ordenada y finita de pasos que resuelve un problema.
Pensamiento computacional	Forma de resolver problemas descomponiéndolos, buscando patrones, abstrayendo y diseñando algoritmos.
Diagrama de flujo	Representación gráfica de un algoritmo con símbolos y flechas.
Lenguaje de programación	Conjunto de reglas y símbolos para escribir programas.
Objeto (sprite)	Personaje o elemento de Scratch con sus propios programas, disfraces y sonidos.
Evento	Suceso que pone en marcha un programa.
Secuencia	Serie de instrucciones que se ejecutan una detrás de otra.
Bucle	Estructura que repite un grupo de instrucciones.
Condicional	Estructura que ejecuta instrucciones según se cumpla o no una condición.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Variable	Espacio de memoria con nombre que guarda un valor que puede cambiar.
Mensaje	Aviso que un objeto envía para que otros reaccionen.
Depurar	Buscar y corregir los errores de un programa.
Actuador	Elemento de salida que actúa sobre el entorno, como un motor o un LED.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Programa

Instrucciones ordenadas para un ordenador.

Algoritmo

Pasos ordenados y finitos para resolver un problema.

Diagrama de flujo

Óvalo inicio, rectángulo acción, rombo decisión.

Scratch

Lenguaje de bloques: escenario, objetos y programas.

Evento

Pone en marcha el programa: bandera, tecla, mensaje.

Secuencia

Instrucciones una tras otra; el orden importa.

Bucle

Repetir n veces, por siempre, repetir hasta que.

Condicional

Si … entonces (… si no).

Operadores

Cálculos, comparaciones, y / o / no, azar.

Variable

Caja con nombre y valor que cambia; inicializarla.

Mensaje

Depurar

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Soluciones

1. **B)** Software.
2. **A)** Como una receta.
3. **D)** Tiene dos salidas: sí o no.
4. **C)** Forma redondeada.
5. **C)** Creado en el MIT.
6. **B)** x de –240 a 240; y de –180 a 180.
7. **A)** Tienen disfraces y sonidos.
8. **D)** Tienen forma de sombrero.
9. **D)** De arriba abajo.
10. **C)** 360 / 4.
11. **B)** 360 / 6.
12. **A)** Típico de los juegos.
13. **A)** Según la condición sea verdadera o falsa.
14. **D)** Bloques hexagonales.
15. **C)** «o» basta con una.
16. **B)** Por ejemplo, los puntos.
17. **B)** Si no, conserva el valor anterior.
18. **A)** Enviar y recibir mensaje.
19. **D)** Los errores se llaman bugs.
20. **C)** Es una salida.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Soluciones de los ejercicios

1. Un programa es un conjunto de instrucciones ordenadas que un ordenador ejecuta para realizar una tarea. Un algoritmo es una secuencia ordenada y finita de pasos que resuelve un problema; el programa es un algoritmo escrito en un lenguaje de programación.
2. Ejemplo, lavarse los dientes: 1) coger el cepillo, 2) poner pasta, 3) abrir el grifo y mojar el cepillo, 4) cepillar dos minutos, 5) enjuagarse, 6) lavar el cepillo y guardarlo.
3. Inicio y fin: óvalo. Acción: rectángulo. Entrada o salida de datos: romboide. Decisión: rombo.
4. Bloques: Scratch y MakeCode. Texto: Python y JavaScript.
5. El escenario es la zona donde ocurre la acción. Los objetos o sprites son los personajes y elementos que se mueven en él, cada uno con sus programas, disfraces y sonidos.
6. Para poner en marcha un programa cuando ocurre algo. Ejemplos: «al hacer clic en la bandera verde» y «al presionar la tecla espacio».
7. Es una estructura que repite instrucciones. En Scratch: repetir n veces, por siempre y repetir hasta que.
8. Una estructura que ejecuta unas instrucciones solo si se cumple una condición (si … entonces), o unas u otras según se cumpla o no (si … entonces … si no).
9. Un espacio de memoria con nombre que guarda un valor que puede cambiar. Ejemplos: puntos, vidas, tiempo o nivel.
10. Buscar y corregir los errores (bugs) para que funcione correctamente.
11. Cuadrado: bajar lápiz, repetir 4 veces mover 80 y girar 90°. Hexágono: repetir 6 veces mover 50 y girar 60° (360 / 6).
12. Inicio → pedir nota (romboide) → ¿nota ≥ 5? (rombo) → sí: mostrar «aprobado» / no: mostrar «suspenso» → fin.
13. Al hacer clic en la bandera: preguntar «¿Qué nota has sacado?» y esperar; si respuesta < 5 entonces decir «Suspenso» si no decir «Aprobado» (Scratch no tiene el bloque ≥, así que se pregunta al revés).
14. Repetir 10 veces hace un número fijo de repeticiones y luego sigue con el programa: dibujar un polígono o dar 10 saltos. Por siempre no termina nunca: comprobar continuamente si el personaje toca un enemigo.
15. «y»: verdadero si se cumplen las dos condiciones (tocando enemigo y vidas = 1). «o»: verdadero si se cumple al menos una (tecla espacio o clic del ratón). «no»: invierte la condición (no tocando suelo → está saltando).
16. Porque si no, conservan el valor de la ejecución anterior: los puntos empezarían, por ejemplo, en 15 en lugar de en 0.
17. Al hacer clic en la bandera: dar a tiempo el valor 30; repetir hasta que tiempo = 0: esperar 1 segundo y sumar –1 a tiempo; decir «¡Se acabó!».

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

19. Copias de un objeto creadas mientras funciona el programa. Son útiles cuando se necesitan muchos elementos iguales: gotas de lluvia, estrellas, disparos o enemigos.
20. Ejecutar por partes haciendo clic en los bloques, leer el código paso a paso en voz alta, mostrar las variables, añadir bloques «decir» para ver por dónde va y cambiar una sola cosa cada vez.
21. Los sensores son entradas que recogen información del entorno (botones, luz, temperatura, acelerómetro). Los actuadores son salidas que actúan sobre el entorno (LED, pantalla, zumbador, motores).
22. Descomponer: invitaciones, comida, música, juegos. Patrones: cada invitación lleva lo mismo. Abstracción: lo importante es la fecha, el lugar y el número de invitados. Algoritmo: los pasos ordenados con fechas.
23. Cesta: al pulsar flechas, cambiar x en ±10. Manzana: al hacer clic en la bandera, dar a puntos 0 y a vidas 3; por siempre: ir a x aleatoria e y = 180; repetir hasta que tocando cesta o y < −170: cambiar y en −5; si tocando cesta sumar 1 a puntos, si no sumar −1 a vidas; si vidas = 0 enviar «fin del juego».
24. Dar a aciertos 0; repetir 5 veces: dar a a un número al azar entre 1 y 10, dar a b otro; preguntar unir(a, «×», b); si respuesta = a · b, sumar 1 a aciertos y decir «Bien», si no decir el resultado correcto; al final decir unir(«Aciertos: », aciertos).
25. Inicio → [bucle] encender verde → esperar 5 s → apagar verde, encender ámbar → esperar 2 s → apagar ámbar, encender rojo → esperar 5 s → apagar rojo → volver al principio (flecha hacia arriba, sin fin).
26. Usa «dar a puntos el valor 1» en lugar de «sumar 1 a puntos». Otro problema: mientras el objeto toque la estrella, sumará muchas veces seguidas; hay que mover la estrella a otro sitio o esconderla después de sumar, o esperar hasta que deje de tocarla.
27. Por siempre: si (temperatura > 17) y (temperatura < 26) entonces mostrar icono feliz, si no mostrar icono triste; esperar 1 segundo. Opcional: al pulsar el botón A, mostrar el número de la temperatura.
28. Porque así cada parte es más fácil de entender, probar y corregir; las partes se pueden reutilizar varias veces sin copiarlas; y varias personas pueden trabajar a la vez. Es aplicar la descomposición del pensamiento computacional.
29. Videojuego: entradas, teclas y ratón; proceso, mover personajes, comprobar colisiones y puntos; salidas, imagen y sonido. Robot aspirador: entradas, sensores de choque, de distancia y de suciedad; proceso, decidir hacia dónde ir; salidas, motores de las ruedas y del cepillo.
30. Pueden tener sesgos si se han diseñado o entrenado con datos injustos; pueden manipular nuestra atención para que pasemos más tiempo en una aplicación; no siempre explican cómo deciden; y la responsabilidad de sus errores no está clara. Por eso hay que programar de forma responsable, transparente y pensando en todas las personas.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información